

Study Guide: Distributed Systems Communication (RPC and RMI)

This study guide provides a comprehensive review of the communication mechanisms used in distributed systems, focusing specifically on Remote Procedure Calls (RPC) and Remote Method Invocation (RMI).

Short-Answer Quiz

1. What is the fundamental motivation for using Remote Procedure Calls (RPC) in distributed systems?
2. Describe the role of the "stub" in an RPC environment.
3. Explain why parameter marshalling involves more than just wrapping data into a message.
4. What are "copy in/copy out" semantics, and how do they relate to RPC?
5. How does a "deferred synchronous RPC" differ from a standard synchronous RPC?
6. In the context of the Distributed Computing Environment (DCE), what is the purpose of the DCE daemon?
7. What are the primary steps a server takes during an RMI to ensure a referenced object is active?
8. Explain the "object-by-value" parameter passing mechanism in RMI and its primary drawback.
9. What information is typically contained within a system-wide object reference?
10. Compare the underlying protocols used by Windows Communication Foundation (WCF) and gRPC.

Answer Key

1. **Fundamental Motivation:** Application developers are already familiar with the simple procedure model where well-engineered procedures operate as isolated "black boxes." RPC allows communication between a caller and a callee to be hidden by the procedure-call mechanism, as there is no fundamental reason procedures cannot execute on separate machines.
2. **Role of the Stub:** The stub acts as a proxy for the remote procedure or object; the client stub builds the request message and calls the local OS, while the server stub unpacks parameters to call the actual server implementation. On the return trip, the server stub

packs the results into a message, and the client stub unpacks the final result to return it to the client application.

3. **Parameter Marshalling:** Marshalling is necessary because client and server machines may utilize different data representations, such as varying byte ordering. It requires transforming values into a sequence of bytes based on an agreed-upon encoding for both basic data (integers, floats) and complex data (arrays, unions) so both machines can properly interpret the message.
4. **Copy In/Copy Out Semantics:** RPC assumes that while a procedure is executed, no assumptions can be made about the parameter values, and all data operated on must be passed as parameters. This model generally excludes passing references to global data, which is why full access transparency is difficult to realize in RPC.
5. **Deferred Synchronous RPC:** While a standard synchronous RPC requires the client to wait for a result, an asynchronous RPC allows the client to continue without waiting for an answer. In a deferred synchronous variation, the client calls the remote procedure and waits for an acceptance acknowledgment before continuing, later receiving the results via a separate one-way RPC or interrupt.
6. **DCE Daemon:** The DCE daemon is used to facilitate client-to-server binding by managing an end-point table on the server machine. A client machine asks the daemon for a specific end-point after looking up the server via a directory server, allowing the client to then perform the actual RPC.
7. **Ensuring Object Activity:** When a method is invoked at the skeleton, the server must ensure the object is active by either creating a separate process to hold the object or loading the object directly into an existing server process. Once active, the request is unmarshaled and the referenced method is invoked.
8. **Object-by-Value:** This mechanism allows a client to pass a complete object as a parameter by marshalling both its state and its methods (or a reference to the method implementation). The primary drawback is that the server creates a copy of the original object, which can introduce complex management problems.
9. **System-wide Object References:** These references generally contain the server address, the port to which adapters listen, and a local object ID. Additionally, they may contain information regarding the specific protocol used for communication, such as TCP, UDP, SOAP, or JSON.
10. **Protocol Comparison:** Windows Communication Foundation (WCF) is a Microsoft-based technology that is primarily Web Services-based, supporting .NET languages. In contrast, gRPC is language-independent and utilizes a binary underlying protocol known as ProtoBuf (Protocol Buffers) from Google.

Essay Questions

1. **The Transparency Dilemma:** Analyze why full access transparency cannot be realized in a standard RPC model. Discuss how the introduction of remote reference mechanisms attempts to enhance this transparency.

2. **The Lifecycle of an RPC:** Detail the ten-step process of a basic RPC operation, from the initial client procedure call to the final return of results. Explain how the involvement of the OS and the network is abstracted away from the developer.
 3. **RMI vs. RPC:** Compare and contrast Remote Procedure Calls and Remote Method Invocation. Focus on the differences in parameter passing, the role of skeletons versus stubs, and how objects are handled in a distributed environment.
 4. **Evolution of Distributed Communication:** Evaluate the characteristics of CORBA, WCF, and gRPC. Discuss how these technologies have evolved to support multi-language environments and different architectural requirements.
 5. **Local vs. Remote Procedure Mechanics:** Using the concept of stack pointers and local variables, explain the mechanical differences between a conventional local procedure call and an RPC. How does the RPC system "mimic" the stack behavior of a local call?
-

Glossary of Key Terms

Term	Definition
Asynchronous RPC	A type of RPC where the client continues execution immediately after sending a request, without waiting for the server's reply.
Client Stub	A local proxy on the client machine that packs parameters into a message and interfaces with the local Operating System to initiate a remote call.
CORBA	A legacy distributed computing technology that provides multi-language support (e.g., Ada, Lisp, Java) for object request brokering.
DCE (Distributed Computing Environment)	A framework that uses daemons and directory servers to help clients locate and bind to server end-points.

gRPC	A language-independent, multi-platform framework that uses Google's ProtoBuf binary protocol for efficient communication.
IDL (Interface Definition Language)	A specialized language used to define the interface between client and server, which is then compiled into stubs and headers.
Marshalling	The process of transforming a data value or object into a sequence of bytes suitable for transmission across a network.
Object-by-Value	An RMI parameter-passing technique where the entire state and implementation of an object are copied from the client to the server.
ProtoBuf	A binary underlying protocol used by gRPC for serializing structured data.
RMI (Remote Method Invocation)	A distributed object mechanism that allows a thread to invoke a method on a remote object using stubs and skeletons.
RPC (Remote Procedure Call)	A protocol that allows a program to execute a procedure in a different address space (usually on another machine) as if it were a local call.
Server Skeleton	A server-side entity that unmarshals incoming RMI requests, ensures the target object is active, and invokes the requested method.

WCF (Windows Communication Foundation)	A Microsoft-based, Web Services-driven architecture for building distributed applications within the .NET framework.
---	--
